

K6 부하 테스트 정리

Cursor 방식 테스트

K6

```
PS C:\dev> k6 run load-test.js

      /W      |--| /--/  /--/
      /W / W   | | /  /  / /
      / W/    W   |  (  /  --W
      /        W   | |W W | ( ) |
      / _____ W |__| W_W W____/ .io

execution: local
script: load-test.js
output: -

scenarios: (100.00%) 1 scenario, 50 max VUs, 1m0s max duration (incl. graceful stop):
  * default: 50 looping VUs for 30s (gracefulStop: 30s)

running (0m30.8s), 00/50 VUs, 1500 complete and 0 interrupted iterations
default ✓ [=====] 50 VUs 30s

✓ status is 200

checks.....: 100.00% ✓ 1500      x 0
data_received.....: 1.8 MB 57 kB/s
data_sent.....: 159 kB 5.2 kB/s
http_req_blocked.....: avg=112.85µs min=0s med=0s max=5.58ms p(90)=0s p(95)=0s
http_req_connecting.....: avg=34.45µs min=0s med=0s max=2.21ms p(90)=0s p(95)=0s
✓ http_req_duration.....: avg=18.24ms min=561.3µs med=8.53ms max=214.98ms p(90)=26.21ms p(95)=70.44ms
  { expected_response:true }...: avg=18.24ms min=561.3µs med=8.53ms max=214.98ms p(90)=26.21ms p(95)=70.44ms
✓ http_req_failed.....: 0.00% ✓ 0      x 1500
http_req_receiving.....: avg=284.31µs min=0s med=0s max=28.16ms p(90)=587.76µs p(95)=986.44µs
http_req_sending.....: avg=51.85µs min=0s med=0s max=4.08ms p(90)=0s p(95)=509.07µs
http_req_tls_handshaking.....: avg=0s min=0s med=0s max=0s p(90)=0s p(95)=0s
http_req_waiting.....: avg=17.9ms min=534.4µs med=8.26ms max=214.98ms p(90)=25.6ms p(95)=70.44ms
http_reqs.....: 1500 48.755705/s
iteration_duration.....: avg=1.02s min=1s med=1.01s max=1.22s p(90)=1.03s p(95)=1.07s
iterations.....: 1500 48.755705/s
vus.....: 50 min=50 max=50
vus_max.....: 50 min=50 max=50
```

PS C:\dev> k6 run load-test.js

```
      /W      |--| /--/  /--/
      /W / W   | | /  /  / /
      / W/    W   |  (  /  --W
      /        W   | |W W | ( ) |
      / _____ W |__| W_W W____/ .io
```

execution: local

script: load-test.js

output: -

scenarios: (100.00%) 1 scenario, 50 max VUs, 1m0s max duration (incl. graceful stop):

* default: 50 looping VUs for 30s (gracefulStop: 30s)

running (0m30.8s), 00/50 VUs, 1500 complete and 0 interrupted iterations

default ✓ [=====] 50 VUs 30s

✓ status is 200

checks.....: 100.00% ✓ 1500 X 0

data_received.....: 1.8 MB 57 kB/s

data_sent.....: 159 kB 5.2 kB/s

http_req_blocked.....: avg=112.85µs min=0s med=0s
max=5.58ms p(90)=0s p(95)=0s

http_req_connecting.....: avg=34.45µs min=0s med=0s
max=2.21ms p(90)=0s p(95)=0s

✓ http_req_duration.....: avg=18.24ms min=561.3µs med=8.53ms
max=214.98ms p(90)=26.21ms p(95)=70.44ms

{ expected_response:true }...: avg=18.24ms min=561.3µs med=8.53ms
max=214.98ms p(90)=26.21ms p(95)=70.44ms

✓ http_req_failed.....: 0.00% ✓ 0 X 1500

http_req_receiving.....: avg=284.31µs min=0s med=0s
max=28.16ms p(90)=587.76µs p(95)=986.44µs

http_req_sending.....: avg=51.85µs min=0s med=0s
max=4.08ms p(90)=0s p(95)=509.07µs

```

http_req_tls_handshaking.....: avg=0s      min=0s      med=0s      max=0s
p(90)=0s      p(95)=0s

http_req_waiting.....: avg=17.9ms   min=534.4µs med=8.26ms
max=214.98ms p(90)=25.6ms   p(95)=70.44ms

http_reqs.....: 1500    48.755705/s

iteration_duration.....: avg=1.02s   min=1s      med=1.01s   max=1.22s
p(90)=1.03s   p(95)=1.07s

iterations.....: 1500    48.755705/s

vus.....: 50      min=50      max=50

vus_max.....: 50      min=50      max=50

```

내용 정리

1. 처리량 제한의 원인: sleep(1) 코드 (생각 시간)

http_req_duration과 iteration_duration을 비교해 보면 흥미로운 점을 발견할 수 있습니다.

- **http_req_duration (실제 API 응답 시간):** 평균 **18.24ms**로 매우 빠릅니다. 성능이 가장 느린 편에 속하는 상위 5%(\$p(95)\$) 요청조차도 **70.44ms** 밖에 걸리지 않았습니다.
- **iteration_duration (반복문 1회 도는 데 걸린 시간):** 평균 **1.02초**, 최솟값은 정확히 **1초**입니다.

핵심 요약: 가상 사용자(VU)들이 API를 호출하는 시간은 0.018초밖에 안 되는데, 정작 다음 요청을 보낼 때까지 약 1초 동안 가만히 기다리고 있다는 뜻입니다. load-test.js 스크립트 마지막에 sleep(1); 코드가 들어가 있을 확률이 매우 높습니다.

이 때문에 50명의 VU가 30초 동안 초당 약 1회씩만 요청을 보내게 되어, 최대 처리량이 초당 50건 근처(**48.75 reqs/s**)로 제한된 것입니다.

2. 시스템 안정성 및 통신 효율

- **에러율: 0.00%** (http_req_failed). 1,500번의 요청 모두 실패 없이 성공(200 OK)했습니다.

- **네트워크 오버헤드:** http_req_connecting(평균 34.45μs)과 http_req_tls_handshaking(0s) 값이 거의 제로에 가깝습니다. k6가 기존 커넥션을 아주 잘 재사용하고 있거나(Keep-Alive), SSL이 없는 로컬/사설 네트워크 환경에서 테스트를 진행하신 것으로 보입니다.
- **서버 처리 특징:** 전체 응답 시간(18.24ms) 중 대부분의 시간인 **17.9ms**가 http_req_waiting(서버가 첫 바이트를 보낼 때까지 걸린 시간)에 소요되었습니다. 즉, 데이터를 주고받는 전송 시간보다는 서버가 비즈니스 로직을 처리하는 데 대부분의 시간을 썼다는 뜻입니다.

다음 단계로 추천하는 액션

현재 서버는 전혀 힘들어하지 않고 여유로운 상태입니다. 시스템의 진짜 한계를 테스트해 보고 싶으시다면 아래 방법을 시도해 보세요.

1. **sleep 제거 또는 단축:** 스크립트 내부의 sleep(1)을 주석 처리하여 가상 사용자들이 쉬지 않고 요청을 퍼붓도록 설정합니다.
2. **가상 사용자(VU) 증가:** VU를 200명, 500명으로 점진적으로 늘려가면서, 응답 시간(\$p(95)\$)이 급격히 늘어나거나 에러가 발생하는 임계점(Breaking Point)을 찾아보세요.

Offset 방식 테스트

```
PS C:\Wdev> k6 run load-test.js

      /W      |--| /--/  /--/
     /W/      W      |  W  W  | (-) |
    / _____ W  |__| W_W W____/ .io

execution: local
script: load-test.js
output: -

scenarios: (100.00%) 1 scenario, 50 max VUs, 1m0s max duration (incl. graceful stop):
  * default: 50 looping VUs for 30s (gracefulStop: 30s)

running (0m30.6s), 00/50 VUs, 1500 complete and 0 interrupted iterations
default ✓ [=====] 50 VUs 30s

✓ status is 200

checks.....: 100.00% ✓ 1500      x 0
data_received.....: 1.7 MB  55 kB/s
data_sent.....: 159 kB  5.2 kB/s
http_req_blocked.....: avg=9.24µs min=0s med=0s max=2.28ms p(90)=0s p(95)=0s
http_req_connecting.....: avg=0s min=0s med=0s max=0s p(90)=0s p(95)=0s
✓ http_req_duration.....: avg=12.83ms min=511.6µs med=11.07ms max=70.63ms p(90)=18ms p(95)=29.23ms
  { expected_response:true }...: avg=12.83ms min=511.6µs med=11.07ms max=70.63ms p(90)=18ms p(95)=29.23ms
✓ http_req_failed.....: 0.00% ✓ 0      x 1500
http_req_receiving.....: avg=182.2µs min=0s med=0s max=8.81ms p(90)=512.62µs p(95)=679.65µs
http_req_sending.....: avg=45.61µs min=0s med=0s max=3.46ms p(90)=0s p(95)=0s
http_req_tls_handshaking.....: avg=0s min=0s med=0s max=0s p(90)=0s p(95)=0s
http_req_waiting.....: avg=12.6ms min=511.6µs med=10.93ms max=70.63ms p(90)=17.93ms p(95)=28.38ms
http_reqs.....: 1500 49.082985/s
iteration_duration.....: avg=1.01s min=1s med=1.01s max=1.08s p(90)=1.02s p(95)=1.03s
iterations.....: 1500 49.082985/s
vus.....: 50 min=50 max=50
vus_max.....: 50 min=50 max=50
```

PS C:\Wdev> k6 run load-test.js

```
      /W      |--| /--/  /--/
     /W /  W      |  W  W  | (-) |
    /  W/      W      |  (  /  --W
   /           W      |  |W  W  | (-) |
  / _____ W  |__| W_W W____/ .io
```

execution: local

script: load-test.js

output: -

scenarios: (100.00%) 1 scenario, 50 max VUs, 1m0s max duration (incl. graceful stop):

* default: 50 looping VUs for 30s (gracefulStop: 30s)

running (0m30.6s), 00/50 VUs, 1452 complete and 0 interrupted iterations

default ✓ [=====] 50 VUs 30s

✓ status is 200

checks.....: 100.00% ✓ 1452 X 0

data_received.....: 1.6 MB 53 kB/s

data_sent.....: 164 kB 5.4 kB/s

http_req_blocked.....: avg=119.06µs min=0s med=0s max=11.02ms
p(90)=0s p(95)=0s

http_req_connecting.....: avg=34.32µs min=0s med=0s max=4.41ms
p(90)=0s p(95)=0s

✓ http_req_duration.....: avg=47.4ms min=1.54ms med=10.51ms max=1.11s
p(90)=20.29ms p(95)=27.15ms

{ expected_response:true }...: avg=47.4ms min=1.54ms med=10.51ms
max=1.11s p(90)=20.29ms p(95)=27.15ms

✓ http_req_failed.....: 0.00% ✓ 0 X 1452

http_req_receiving.....: avg=133.97µs min=0s med=0s max=10.7ms
p(90)=530.53µs p(95)=693.33µs

http_req_sending.....: avg=114.31µs min=0s med=0s max=6.09ms
p(90)=0s p(95)=625.64µs

http_req_tls_handshaking.....: avg=0s min=0s med=0s max=0s
p(90)=0s p(95)=0s

http_req_waiting.....: avg=47.15ms min=599.5µs med=10.36ms max=1.11s
p(90)=19.96ms p(95)=26.42ms

http_reqs.....: 1452 47.435537/s
 iteration_duration.....: avg=1.05s min=1s med=1.01s max=2.12s
 p(90)=1.02s p(95)=1.03s
 iterations.....: 1452 47.435537/s
 vus.....: 50 min=50 max=50
 vus_max.....: 50 min=50 max=50

지표	이전 테스트 (Test 1)	현재 테스트 (Test 2)	상태 변화
총 요청 수 (주 처리량)	1,500회 (~48.7 req/s)	1,452회 (~47.4 req/s)	소폭 감소 (지연의 영향)
에러율 (http_req_failed)	0.00% (0건)	0.00% (0건)	완벽 유지
평균 응답 속도 (avg)	18.24ms	47.4ms	약 2.5배 느려짐
중앙값 (med, 50%의 속도)	8.53ms	10.51ms	거의 비슷 (여전히 빠름)
상위 95% 속도 (p(95))	70.44ms	27.15ms	오히려 개선됨!
최대 지연 시간 (max)	214.98ms	1.11s (1,110ms)	대폭 증가 (1초 돌파)

결론 :

이번 테스트에서는 Offset 평균 응답 시간이 더 높게 나왔지만, 데이터 수가 적어서 성능 차이를 확정하긴 어렵고, 10만 건 이상 데이터에서 OFFSET 값을 크게 주고 비교하면 더 성능 차이가 있다.